REAL TIME OPERATING SYSTEM

RTOS LAB

Assignment 3

Name: **Varun B Karmikanda**

USN: 261100610029

Program: ME Computer Science & Engineering

November 4, 2026

1. Shared pthread

Question: Create two threads that access shared and local variables, and display their process and thread information.

Code:

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

int shared = 100;

static void *worker(void *arg)
{
    int mine = 1;
    pid_t pid = getpid();
    pthread_t me = pthread_self();

    (void)arg;

    printf("pid=%d thread=%lu\n", (int)pid, (unsigned long)me);
    printf("&shared=%p &mine=%p\n",
           (void *)&shared, (void *)&mine);

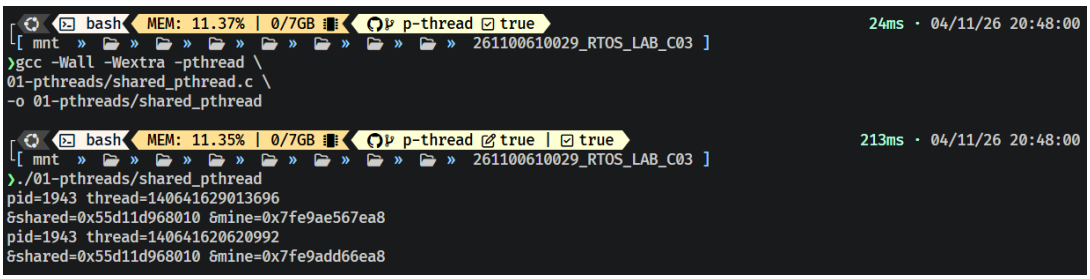
    sleep(1);
    return NULL;
}

int main(void)
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```



2. Basic Fork

Question: Create a child process using `fork()` and display the parent and child process IDs.

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Child process
        printf("Child: my pid = %d, parent pid = %d\n", getpid(), getppid());
    } else {
        // Parent process
        printf("Parent: my pid = %d, child pid = %d\n", getpid(), pid);
        wait(NULL);  // wait for child to finish
        printf("Parent: child has exited\n");
    }

    return 0;
}
```

```
 bash  MEM: 11.35% | 0/7GB  p-thread true | true                    1.016s · 04/11/26 20:49:00
[ mnt » » » » » » » » » 261100610029_RTOS_LAB_C03 ]
❯gcc -Wall -Wextra \
02-basic-fork/basic_fork.c \
-o 02-basic-fork/basic_fork

 bash  MEM: 11.59% | 0/7GB  p-thread true | true                    173ms · 04/11/26 20:51:00
[ mnt » » » » » » » » » 261100610029_RTOS_LAB_C03 ]
❯./02-basic-fork/basic_fork
Parent: my pid = 2314, child pid = 2315
Child: my pid = 2315, parent pid = 2314
Parent: child has exited
```

3. Fork With Sleep

Question: Create a child process that sleeps for 20 seconds while the parent waits for it using `waitpid()`.

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Child process
        printf("Child: my pid = %d, parent pid = %d\n", getpid(), getppid());
        printf("Child: sleeping for 20 seconds...\n");
        fflush(stdout);
        sleep(20);
        printf("Child: woke up, exiting now\n");
    } else {
        // Parent process
        printf("Parent: my pid = %d, child pid = %d\n", getpid(), pid);
        printf("Parent: waiting for child to finish...\n");
        fflush(stdout);

        int status;
        pid_t exited_pid = waitpid(pid, &status, 0);  // blocks here for ~20 sec

        if (exited_pid == pid) {
            if (WIFEXITED(status))
                printf("Parent: child %d exited with status %d\n", exited_pid,
                    WEXITSTATUS(status));
            else
                printf("Parent: child %d did not exit normally\n", exited_pid);
        }
    }

    return 0;
}
```

```
bash MEM: 11.64% | 0/7GB p-thread true | true                    14ms · 04/11/26 20:51:00
[ mnt » » » » » » » » » » 261100610029_RTOS_LAB_C03 ]
❯gcc -Wall -Wextra \
03-fork-with-sleep/fork_sleep.c \
-o 03-fork-with-sleep/fork_sleep

bash MEM: 11.58% | 0/7GB p-thread true | true                    172ms · 04/11/26 20:52:00
[ mnt » » » » » » » » » » 261100610029_RTOS_LAB_C03 ]
❯./03-fork-with-sleep/fork_sleep
Parent: my pid = 2651, child pid = 2652
Parent: waiting for child to finish...
Child: my pid = 2652, parent pid = 2651
Child: sleeping for 20 seconds...
Child: woke up, exiting now
Parent: child 2652 exited with status 0
```

4. Fork Three Children

Question: Create three child processes with different sleep durations and wait for all of them.

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int sleep_times[3] = {10, 20, 30};
    pid_t child_pids[3];

    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork failed");
            return 1;
        } else if (pid == 0) {
            // Child process
            printf("Child %d: pid = %d, sleeping for %d seconds\n",
                   i, getpid(), sleep_times[i]);
            fflush(stdout);
            sleep(sleep_times[i]);
            printf("Child %d: pid = %d, woke up after %d seconds\n",
                   i, getpid(), sleep_times[i]);
            return 0;  // child exits here, doesn't continue the loop
        } else {
            // Parent process: record child's pid and move on to fork the next one
            child_pids[i] = pid;
        }
    }

    // Parent waits for all 3 children
    printf("Parent: pid = %d, waiting for all children...\n", getpid());
    fflush(stdout);

    for (int i = 0; i < 3; i++) {
        int status;
        pid_t exited_pid = waitpid(child_pids[i], &status, 0);
        if (exited_pid == child_pids[i] && WIFEXITED(status)) {
            printf("Parent: child %d (pid %d) exited with status %d\n",
                   i, exited_pid, WEXITSTATUS(status));
        }
    }

    printf("Parent: all children have exited\n");
    return 0;
}
```

```
bash MEM: 11.63% | 0/7GB p-thread true | true 20.019s · 04/11/26 20:53:00
[ mnt » » » » » » » » » » 261100610029_RTOS_LAB_C03 ]
❯gcc -Wall -Wextra \
04-multiple-children/fork_three_children.c \
-o 04-multiple-children/fork_three_children

bash MEM: 11.69% | 0/7GB p-thread true | true 208ms · 04/11/26 20:55:00
[ mnt » » » » » » » » » » 261100610029_RTOS_LAB_C03 ]
❯./04-multiple-children/fork_three_children
Child 0: pid = 3108, sleeping for 10 seconds
Parent: pid = 3107, waiting for all children...
Child 1: pid = 3109, sleeping for 20 seconds
Child 2: pid = 3110, sleeping for 30 seconds
Child 0: pid = 3108, woke up after 10 seconds
Parent: child 0 (pid 3108) exited with status 0
Child 1: pid = 3109, woke up after 20 seconds
Parent: child 1 (pid 3109) exited with status 0
Child 2: pid = 3110, woke up after 30 seconds
Parent: child 2 (pid 3110) exited with status 0
Parent: all children have exited
```

5. Fork and waitpid

Question: Use `waitpid()` to wait specifically for the child created by the parent.

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Child process
        printf("Child: my pid = %d, parent pid = %d\n", getpid(), getppid());
    } else {
        // Parent process
        printf("Parent: my pid = %d, child pid = %d\n", getpid(), pid);

        int status;
        pid_t exited_pid = waitpid(pid, &status, 0);  // wait specifically for this child's pid

        if (exited_pid == pid) {
            if (WIFEXITED(status))
                printf("Parent: child %d exited with status %d\n", exited_pid, WEXITSTATUS(status));
            else
                printf("Parent: child %d did not exit normally\n", exited_pid);
        }
    }

    return 0;
}
```

```
bash MEM: 11.66% | 0/7GB p-thread true | true
[ mnt » » » » » » » » » » 261100610029_RTOS_LAB_C03 ]
❯gcc -Wall -Wextra \
05-waitpid/fork_waitpid.c \
-o 05-waitpid/fork_waitpid

bash MEM: 11.84% | 0/7GB p-thread true | true                    202ms · 04/11/26 20:57:00
[ mnt » » » » » » » » » » 261100610029_RTOS_LAB_C03 ]
❯./05-waitpid/fork_waitpid
Parent: my pid = 3420, child pid = 3421
Child: my pid = 3421, parent pid = 3420
Parent: child 3421 exited with status 0
```

6. Recursive Fork

Question: Create a chain of processes where each child creates another child, and use `waitpid()` between parent and child processes.

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("Original parent: pid = %d\n", getpid());
    fflush(stdout);

    pid_t pid1 = fork();

    if (pid1 < 0) {
        perror("fork failed");
        return 1;
    } else if (pid1 == 0) {
        // ---- child1 ----
        printf("Child1: pid = %d, parent pid = %d\n", getpid(), getppid());
        fflush(stdout);

        pid_t pid2 = fork();

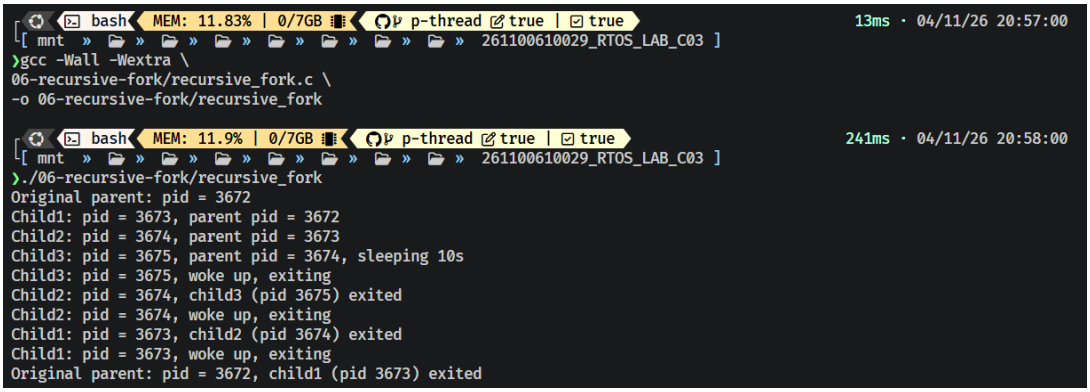
        if (pid2 < 0) {
            perror("fork failed");
            return 1;
        } else if (pid2 == 0) {
            // ---- child2 ----
            printf("Child2: pid = %d, parent pid = %d\n", getpid(), getppid());
            fflush(stdout);

            pid_t pid3 = fork();

            if (pid3 < 0) {
                perror("fork failed");
                return 1;
            } else if (pid3 == 0) {
                // ---- child3 ----
                printf("Child3: pid = %d, parent pid = %d, sleeping 10s\n", getpid(),
                    getppid());
                fflush(stdout);
                sleep(10);
                printf("Child3: pid = %d, woke up, exiting\n", getpid());
                return 0;
            } else {
                // child2 waits for child3
                int status;
                waitpid(pid3, &status, 0);
                printf("Child2: pid = %d, child3 (pid %d) exited\n", getpid(), pid3);
                sleep(10);
                printf("Child2: pid = %d, woke up, exiting\n", getpid());
                return 0;
            }
        } else {
            // child1 waits for child2
            int status;
            waitpid(pid2, &status, 0);
            printf("Child1: pid = %d, child2 (pid %d) exited\n", getpid(), pid2);
            sleep(10);
            printf("Child1: pid = %d, woke up, exiting\n", getpid());
            return 0;
        }
    } else {
        // ---- original parent ----
        int status;
        waitpid(pid1, &status, 0);
        printf("Original parent: pid = %d, child1 (pid %d) exited\n", getpid(), pid1);
    }

```

```
66      return 0;
67  }
```



7. Fork and Exec

Question: Create a child using `fork()` and replace the child program using `execl()`.

Code – fork_exec.c:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("fork_exec: parent pid = %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Child process: about to become exec_prog
        printf("fork_exec: child pid = %d, about to exec\n", getpid());
        fflush(stdout);

        execl("./exec_prog", "exec_prog", NULL);

        // only reached if execl fails
        perror("execl failed");
        return 1;
    } else {
        // Parent waits for the child (now running as exec_prog)
        int status;
        waitpid(pid, &status, 0);
        printf("fork_exec: child (pid %d) exited\n", pid);
    }

    return 0;
}
```

Code – exec_program.c:

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("exec_prog: pid = %d, starting count\n", getpid());
    fflush(stdout);

    for (int i = 1; i <= 10; i++) {
        printf("%d\n", i);
        fflush(stdout);
        sleep(1);  // slows it down so you have time to inspect it while running
    }

    return 0;
}
```


PS C:\Users\VarunKarmikanda\msis\sem1\rtos> wsl
bash MEM: 11.8% | 0/7GB p-thread true | true
30.021s · 04/11/26 20:59:00
[mnt » » » » » » » » » 261100610029_RTOS_LAB_C03]
❯gcc -Wall -Wextra \
07-fork-and-exec/exec_program.c \
-o 07-fork-and-exec/exec_prog
bash MEM: 11.79% | 0/7GB p-thread true | true
201ms · 04/11/26 21:00:00
[mnt » » » » » » » » » 261100610029_RTOS_LAB_C03]
❯gcc -Wall -Wextra \
07-fork-and-exec/fork_exec.c \
-o 07-fork-and-exec/fork_exec
bash MEM: 11.8% | 0/7GB p-thread true | true
183ms · 04/11/26 21:00:00
[mnt » » » » » » » » » 261100610029_RTOS_LAB_C03]
❯cd 07-fork-and-exec
./fork_exec
fork_exec: parent pid = 4027
fork_exec: child pid = 4028, about to exec
exec_prog: pid = 4028, starting count
1
2
3
4
5
6
7
8
9
10
fork_exec: child (pid 4028) exited